Report

1. Assignment Overview

The notebook implements a deep reinforcement learning workflow centered on policy gradient methods for the BipedalWalker-v3 control task. The overall objective is to train an agent that can interact with the continuous state and continuous action environment, improve its policy through on-policy rollouts, and then export the learned policy for submission and visualization.

2. Notebook Structure Analysis

The first code block detects the host operating system, disables the Colab-specific execution path, and sets CUDA_VISIBLE_DEVICES to select the first GPU when available. The import block then loads PyTorch, Gymnasium, ONNX, onnx2pytorch, NumPy, tqdm, matplotlib, seaborn, and notebook display utilities. A Linux-only virtual display is started through pyvirtualdisplay so that Gym rendering and video capture remain usable in headless environments.

2.1 Environment utilities and evaluation helpers

The helper functions wrap the Gymnasium environment with episode statistics recording and optional video capture. The environment creation function instantiates BipedalWalker-v3 with a maximum episode length of 1600 steps, render_mode='rgb_array', and an optional hardcore flag. This is a deliberate choice because the notebook needs both a training environment and a visual evaluation environment that can emit frames for saving to disk.

2.2 Logging and monitoring

The Logger class stores run specific artifacts under a timestamped directory. It creates separate folders for logs and videos, tracks optional gradient statistics, supports log dumping to CSV-style files, and can copy a parameter file into the run directory when provided. This design supports offline debugging and later inspection of training runs.

2.3 Trajectory storage and return/advantage computation

The notebook defines Transition and Episode helper classes to store per-step data: state, action, reward, next state, and log-probabilities. A BufferDataset wrapper converts the flattened rollout buffer into a PyTorch DataLoader. The RolloutBuffer class stores complete episodes first and then flattens them after computing returns and Generalized Advantage Estimation (GAE).

2.4 optimized training loop

The later notebook section introduces a separate training pipeline built around a vectorized SyncVectorEnv, RunningMeanStd statistics for observations and returns, and a more explicit PPO loop.

This block collects rollouts, normalizes observations and rewards, computes GAE, and calls ppo_update followed by periodic evaluation. The checkpoint with the best evaluation reward is stored as best_model.pt.

2.9 Visualization and export

The notebook ends with result plotting, average reward calculation, checkpoint reloading, ONNX export, optional file download in Colab, visualization of the agent in the environment, and a demo evaluation using a loaded ONNX model. These blocks are intended to prove that the trained policy can be saved, reloaded, and executed outside the training loop.

3. Local Machine Adaptation

Several changes in the notebook clearly target local execution rather than a pure Colab workflow. First, Colab detection is disabled by setting IN_COLAB = False. Second, GPU handling is made explicit by setting CUDA_VISIBLE_DEVICES='0' and selecting torch.device('cuda' if available else 'cpu'). Third, a Linux virtual display is started so that rendering works without a physical desktop session. Fourth, the environment is configured with render_mode='rgb_array' so that frame capture and video writing are possible in a non-interactive runtime.

These changes are necessary because the original notebook logic depends on notebook rendering, environment video capture, and GPU acceleration. Without those adjustments, Gymnasium video capture and policy evaluation would fail or become impractical on a local headless machine.

The notebook also partially preserves Colab-specific behavior through the file download cell that imports google.colab.files.

4. Implementation Details

The implementation is built around continuous action reinforcement learning with policy gradients. The policy is a Gaussian actor that samples actions from a Normal distribution parameterized by a neural network. The critic approximates the value function needed for advantage estimation. Training follows the PPO family of methods: it uses clipped policy ratios, advantage normalization, a value regression loss, and an entropy bonus for exploration.

The code relies on PyTorch for model definition and optimization, Gymnasium for the environment, ONNX and onnx2pytorch for model export and reloading, NumPy for numeric processing, and Matplotlib/Seaborn for analysis and plotting. The workflow also includes video recording and inline embedding for human verification of policy behavior.

The explicit hyperparameter configuration in the final training loop shows a deliberate optimization strategy: many parallel environments to improve sample throughput, moderate rollout windows, a smaller minibatch size for multiple epochs over each rollout, reward normalization, observation normalization, and learning rate decay over the course of training.

5. Challenges Faced

The notebook shows several implementation challenges that were addressed directly in code. The first is runtime portability: Colab-oriented rendering and file download behavior had to be adapted to local execution. The second is environment rendering: a virtual display is required on Linux systems to avoid video-capture failures in headless contexts.

The third challenge is numerical stability. The actor clamps log-standard-deviation values, adds a small epsilon to the standard deviation, clips actions to the valid range, normalizes advantages, and clips gradients. These steps all mitigate instability common in continuous-control policy-gradient methods.

The fourth challenge is batch-size edge cases. The rollout buffer explicitly uses drop_last=True in the DataLoader to avoid one-sample batches that can create NaN standard deviations during normalization.

A fifth issue is notebook consistency. The visible file contains references to names that are not defined in the shown cells, including Actor, Critic, ppo_update, and evaluate_policy. One cell also contains the stray token 'c# load model from checkpoint', which is syntactically valid but will raise a NameError at runtime because c is undefined. Finally, the Colab download block is not portable to local execution.

6. Improvements and Optimizations

The notebook introduces several meaningful optimizations: observation normalization through RunningMeanStd, reward normalization using a running return statistic, batched rollouts across 16 environments, a decayed learning rate schedule, clipped PPO updates, and explicit gradient clipping. These choices improve learning stability and make the training loop less sensitive to scale differences and exploding gradients.

The code also cleans up the workflow by separating utility functions, environment handling, model definitions, buffer logic, agent logic, and evaluation/visualization steps. This separation makes the notebook easier to audit and to re run in stages.

7. Final Pipeline Summary

Input: raw BipedalWalker-v3 observations from the Gymnasium environment. Processing: normalization of observations, rollout collection, reward scaling, and trajectory storage. Model: a Gaussian actor network and a value critic network trained with a PPO-style objective. Output: sampled actions during interaction, saved checkpoints, an ONNX policy export, and recorded evaluation videos. Evaluation: episodic reward tracking, returns, entropy, checkpoint selection based on evaluation performance, and qualitative video inspection.

8. Lessons Learned

The notebook demonstrates that robust reinforcement learning code is not only about the optimization algorithm. Environment rendering, device selection, file paths, logging,

normalization, and export format compatibility all matter for a successful end-to-end implementation.

It also shows that notebook refactoring can introduce inconsistency if multiple versions of a workflow are merged without reconciling the dependencies.